

University of Colorado Boulder
ASEN 5264

Waypoint Path Planning For the SAFARI System

Bijan Jourabchi
bijan.jourabchi@colorado.edu

Jack Ryan
john.ryan-1@colorado.edu

Parker Graham
parker.graham@colorado.edu

CU-Boulder, Ann & HJ Smead Aerospace Engineering Sciences
April 30, 2026

1 Abstract

The Search Autonomy for Aerial Robotic Intelligence (SAFARI) system employs probabilistic and machine learning techniques to generate a spatial reward map from user-defined inputs [5] [6], which serves as the basis for trajectory planning across small uncrewed aerial systems (sUAS). To enhance operational flexibility, the system supports waypoint-constrained trajectory generation, enabling user-specified spatial constraints to be directly incorporated into the autonomous flight planning pipeline.

We formulate Waypoint Navigation as a finite-horizon MDP $(\mathcal{S}, \mathcal{A}, \mathcal{T}, \mathcal{R}, \gamma)$ over a discretized grid representation of the flight area, where each cell constitutes a discrete state in the planning space. The state space $\mathcal{S}$ encodes the agent’s 2-D grid position along with distance traveled. The reward function $\mathcal{R}$ is defined over a user-provided spatial reward map. The planning pipeline is decomposed into two stages: waypoint ordering and segment-level path planning. Waypoint ordering is performed via approximate solutions to the DISCOUNTED REWARD TRAVELING SALESMAN PROBLEM and the MINIMUM-EXCESS PATH PROBLEM. Segment-level planning is then carried out using MDP algorithms, such as MONTE CARLO TREE SEARCH (MCTS). The objective is to find a policy π^* that maximizes expected cumulative reward while guaranteeing full waypoint coverage.

2 Introduction

Unmanned aerial vehicles (UAVs) are seeing increased use in applications ranging from agriculture and geological surveys to first response. They are beginning to play a pivotal role in many of these fields, including search and rescue (SR) missions conducted by first responders. UAVs used in SR operate in settings where time and coverage efficiency are critical. These missions are inherently uncertain because the target location is unknown and human-defined regions of interest only indicate where a lost person or animal is more likely to be found. In autonomous SR, a human operator may identify rivers, trails, roads, or other relevant terrain features as high-priority areas, which the system must then use to plan an efficient search trajectory.

The SAFARI system addresses this type of search problem by using probabilistic and machine learning techniques to generate a discretized spatial reward map from user-defined regions of interest. In this map, each grid cell is assigned a reward corresponding to its expected search value. High-reward regions represent areas where the user believes the target is more likely to be located, such as rivers, trails, roads, or terrain features associated with likely movement.

We model the search environment as a discrete grid. At each time step, the agent may move to one of the neighboring cells in an 8-connected action space. This converts the search task into a discrete planning problem in which the agent must choose actions that trade off path length, reward collection, waypoint visitation, and obstacle avoidance.

The goal of this work is to provide a coverage efficient SR planning algorithm with which a user can input critical, mission specific parameters. This will begin with optimizing the waypoint ordering in terms of distance and reward regions. Then path planning between each ordered waypoint will be explored. Specific to our case, this will include a user defined detour budget. It defines the number of additional actions the agent may take beyond the shortest path connecting 2 waypoints.

To address these challenges, we propose a two-stage planning framework that combines waypoint ordering with constrained path planning. First, unordered user-specified waypoints are organized into an efficient visitation sequence using an approximation of the Discounted Reward Traveling Salesman Problem (DRTSP), which prioritizes both spatial reward accumulation and traversal efficiency under exponential discounting. Second, given the ordered waypoint sequence, we perform segment-level trajectory generation using a constrained Markov Decision Process formulation over an 8-connected grid. Each method is augmented with a detour budget constraint, which enforces waypoint reachability while allowing controlled deviation from the shortest path to collect reward. Together, these components produce a trajectory that balances waypoint visitation requirements with coverage of high-reward regions in the SAFARI reward map.

3 Methodology

3.1 Waypoint Ordering

In practical settings, users place waypoints in an arbitrary order, with little consideration of an optimal path. This motivates a waypoint ordering state that sorts waypoints in a visiting order before any waypoint to waypoint planning is performed. The full ordering procedure is composed of three components: a distance metric that accounts for obstacles (keep out zones), a guess enumeration framework that breaks the problem into smaller subproblems, and a Minimum Excess Path solver that constructs each path segment.

We formulate waypoint ordering as an instance of the DISCOUNTED REWARD TRAVELING SALESMAN PROBLEM (DR-TSP) introduced in Blum et al. [1]. On an undirected graph $G = (V, E)$, where each node $v \in V$ represents a waypoint and each edge weight represents the traversable distance between nodes in grid space. The goal is to find a path P starting at a fixed source node $s \in V$ that maximizes the total discounted reward collected across all visited waypoints. Each waypoint v carries a prize $\pi(v)$ that decays exponentially with travel distance according to a discount factor $\gamma \in (0, 1)$. For a path P starting at s , let $d^P(u, v)$ denote the distance traveled along P from u to v , and let d^P denote the total path length. The total discounted reward collected by P is then

$$\rho^P = \sum_{v \in P} \pi(v) \gamma^{d^P(s, v)}.$$

The *excess* of a path P from s to t measures how much P deviates from the shortest possible route,

$$\text{excess}(P) = d^P(s, t) - d(s, t).$$

For this application, the prize $\pi(v)$ was chosen to be the reward provided by the user’s spatial reward map summed with the reward from all neighboring cells.

User defined “keep-out zones” in the flight area can block direct routes, thus a more robust distance metric is needed. To compute these distance, we use the well known wavefront path planning approach [3]. The wavefront algorithm is a grid based algorithm that computes the shortest path distances by propagating a “wave” from the goal cell via a breadth-first search (BFS). Each cell is labeled with its distance from the goal cell. When an obstacle is encountered, the wave will propagate around it, ensuring that the resulting distances are consistent with the presence of an obstacle. Algorithm 3 describes the procedure in detail. All subsequent references to $d(u, v)$ will represent distances computed via wavefront generation.

Our proposed solution to DR-TSP follows the structure outlined in Farbstain and Levin [4], which solves the Minimum Excess Path problem as a subroutine to solving the larger problem. Following Algorithm 1 from Farbstain and Levin [4], we begin by rescaling each edge weight so that the discount factor become $\gamma = 1/2$.

$$\hat{d}(u, v) = d(u, v) \cdot \log_2(1/\gamma)$$

We also fix a sequence of parameters $\varepsilon_1, \varepsilon_2, \dots, \varepsilon_\ell$ defined by $\varepsilon_i = -\log_2(x_i)$ where $x = [0.84, 0.62, 0.42]$ and $\ell = 3$, following the configuration in [4]. Since DR-TSP is intractable to solve directly [1], we adopt the guess enumeration framework presented in [4]. The idea is to decompose the unknown optimal path P^* into ℓ substages, and in each substage guess a tuple (t_i, t'_i, u_i, k_i) , where t_i is the last node of P^* whose subpath from the previous stage endpoint t'_{i-1} has excess less than ε_i , t'_i is the node immediately following t_i along P^* , u_i is the node in stage i attaining the highest scaled prize, and k_i approximates the total scaled prize collected in stage i . k_i cannot be computed directly, and is instead estimated via lower and upper bound approximated in Farbstain and Levin [4]. The algorithm will exhaustively enumerate all combinations of guess tuples across all ℓ stages and retains the path achieving the highest true discounted reward.

With a guess tuple fixed for each stage i , the algorithm now needs to construct a path from t'_{i-1} to t_i that collects scaled prize of at least k_i while minimizing its excess. This is known as the Minimum Excess Path (MEP) subproblem. We implement this subroutine as a greedy heuristic, accepting a sub-optimal solution that is sufficient in this application. The heuristic relies on the scaled prize $r_i^{\text{apx}}(v)$, which represents the

prize of node v discounted by the estimated travel cost accumulated through stage i . Formally,

$$r_i^{\text{apx}}(v) = \begin{cases} \pi(v) \cdot 2^{-\hat{d}(s, v)}, & i = 1, \\ \pi(v) \cdot 2^\Phi \cdot 2^\Psi, & i > 1, \end{cases} \quad (1)$$

$$\Phi = -\sum_{j=1}^{i-1} [\hat{d}(t'_{j-1}, t_j) + \hat{d}(t_j, t'_j)] + \hat{d}(t'_{i-1}, v) \quad \Psi = -\sum_{j=1}^{i-1} \varepsilon_j^{\text{apx}}$$

where $\varepsilon_j^{\text{apx}}$ is the recorded excess of the segment returned by the MEP solver in stage j , and any node already visited is assigned $r_i^{\text{apx}}(v) = 0$ [4]. At each step of the heuristic, the unvisited node $v \neq t_i$ that maximizes the reward-per-cost ratio $\frac{r_i^{\text{apx}}(v)}{d(\text{curr}, v)}$ is greedily selected and appended to the path until the threshold k_i is met, after which the path is closed at t_i . The excess $\varepsilon_i^{\text{apx}}$ of the returned segment is then recorded and used to update r_{i+1}^{apx} for the next stage. Algorithm 1 describes this procedure formally.

Algorithm 1 MEP solver

Require: s (Start Node), t (End Node), $\hat{d}$ (Distance Matrix), $r_i^{\text{apx}}(v)$ (Scaled Prize at a Given Node), k_i (Required Accumulated Reward).

```

1: Curr  $\leftarrow s$ 
2: Collected  $\leftarrow r_i^{\text{apx}}(s)$ 
3: Visited( $n$ )  $\leftarrow 0 \quad \forall n \in V \setminus s$  ▷ Keep Track of Visited Nodes
4:  $Q \leftarrow \emptyset$  ▷ Path Segment
5: while Collected  $< k_i$  do
6:    $v_{\text{best}} \leftarrow 0 \quad \rho_{\text{best}} \leftarrow -\infty$ 
7:   for  $v \in \text{keys}(r_i^{\text{apx}})$  do
8:     if  $v \in \text{Visited}$  then continue
9:     end if
10:    cost =  $d(\text{Curr}, v)$ 
11:    score =  $\frac{r_i^{\text{apx}}(v)}{\text{cost}}$ 
12:    if score  $> \rho_{\text{best}}$  then
13:       $\rho_{\text{best}} \leftarrow \text{score} \quad v_{\text{best}} \leftarrow v$ 
14:    end if
15:  end for
16:   $Q \leftarrow Q \cup v_{\text{best}}$ 
17:  Visited( $v_{\text{best}}$ )  $\leftarrow 1$ 
18:  Collected  $\leftarrow \text{Collected} + r_i^{\text{apx}}(v_{\text{best}})$ 
19:  Curr  $\leftarrow v_{\text{best}}$ 
20: end while
21:  $\varepsilon_i^{\text{apx}} = d^Q - \hat{d}(s, t)$  ▷ Compute Excess
22: return ( $Q, \varepsilon_i^{\text{apx}}$ )
```

Note that Algorithm 1 returns a segment Q_i for each stage i , the segments are concatenated in order to form the full candidate path P . If any stage fails to find a feasible segment, that is, the prize threshold k_i cannot be met with the remaining unvisited nodes, the entire guess tuple is discarded. Across all enumerated guess tuples, the candidate path achieving the highest true discounted reward ρ^P is returned as the solution. The complete ordering procedure is summarized in Algorithm 2.

Algorithm 2 DR-TSP Waypoint Ordering

Require: g (Guess Tuples), s (Start Node), $\hat{d}$ (Distance Matrix), V (Set of All Nodes).

```
1:  $P \leftarrow s$ 
2:  $\varepsilon^{\text{apx}} = \emptyset$ 
3: for  $i \in 1 \dots \ell$  do
4:    $t_i, t'_i, u_i, k_i \leftarrow g(i)$   $\triangleright$  Unpack Guess Tuple
5:    $t'_{i-1} \leftarrow \begin{cases} s & \text{if } i = 1 \\ t'_{i-1} & \text{otherwise} \end{cases}$ 
6:    $r_i^{\text{apx}} \leftarrow \text{Follow (1)}$ 
7:    $Q_i, \text{excess} \leftarrow \text{MEP Solver}$ 
8:    $P \leftarrow P \cup Q_i$ 
9:    $\text{push}!(\varepsilon^{\text{apx}}, \text{excess})$ 
10: end for
11: return  $P$ 
```

The returned path P will be the final ordered list of waypoints. It is important to note that Algorithm 2 will work with any MEP solver. Algorithm 2 follows the structure of Algorithm 2 from Farbstein and Levin, with a greedy heuristic replacing the approximate MEP subroutine [4].

3.2 Goal Oriented MCTS

With the broader problem of waypoint ordering being solved, we now turn to waypoint to waypoint planning. We formulate this as a finite-horizon MDP over a discretized grid and solve it using Monte Carlo Tree Search (MCTS). Standard MCTS is not directly applicable here since the planner has no guarantee of reaching the target waypoint. Even giving waypoints an arbitrarily large reward does not guarantee the planner will visit the waypoint. Our design address this through two coupled methods, a detour budget that enforces reachability, and a wavefront guided rollout policy that biases exploration towards the goal.

Each state $s \in \mathcal{S}$ is a tuple (p, g) where p is the agent's current grid position and g is the total distance traveled from the start. At each step, the agent may move to any of the eight neighboring cells with a step cost of 1 for any move. The central mechanism that ensures waypoints are visited is the detour budget B . Let $d(p)$ denote the wavefront distance from a position p to the waypoint. A state (p, g) is declared feasible if and only if $g + d(p) \leq d^*(s, wp) + B$ where $d^*(s, wp)$ is the shortest distance from the start to the waypoint, and B is our detour budget. The left hand side represents the minimum total travel cost the agent would need to incur from its current position, and the right hand side is the maximum total travel permitted. Any state that violates this constraint is pruned from the search. As a result, every path explored by MCTS is consistent with the agent reaching the target waypoint. This approach strictly prohibits the planner from committing to a trajectory where the goal is unreachable. The detour budget B is a tunable exploration parameter. A larger budget allows the agent to deviate further from the direct route.

When MCTS expands its tree into a leaf node, it performs a rollout simulation to estimate the value of that state. Rather than selecting random actions, we use a wavefront guided heuristic that at each step, selects the action that will minimize its distance to the waypoint. This biases rollouts towards the waypoint, producing value estimates that reflect trajectories the agent can actually achieve. This was selected to improve sample efficient compared to random simulation [2].

Node selection within the tree follows the standard UCB architecture. In our case, the action space is dependent on the state, $\mathcal{A}(s)$, and only contains feasible actions. That is, actions that satisfy the detour budget constraint. We handle obstacles by assigning an obstacle cell with an extremely large negative reward, effectively removing it from consideration. To discourage the agent from revisiting cells, a penalty of -100 is applied to the reward of any previously visited cell during simulation. This does not prevent revisits, but makes them relatively unattractive to the planner relative to unvisited, high reward cells.

3.3 Value Iteration

To provide an exact baseline for the waypoint planning, we solve the same finite horizon MDP using Value Iteration (VI). The state and action spaces are identical to those used in the previous section. Each state is defined as $s = (p, g)$, where p is the current grid position and g is the total distance traveled from the start, and the agent may move to any of the eight neighboring cell with a unit step cost. Reachability is enforced using the same detour budget (B) constraint. Let $d(p)$ denote the wavefront distance from position p to the waypoint. A state (p, g) is feasible if and only if $g + d(p) \leq d^*(s, wp) + B$, where $d^*(s, wp)$ is the shortest path distance from the start to the waypoint. Any state that violates this condition is excluded from the state space. That means any resulting policy must reach the target waypoint.

The value function is then computed through standard bellman backups,

$$V^*(s) = \max_{a \in A(s)} \left[r(s, a) + \gamma \sum_{s' \in S} T(s, a, s') V^*(s') \right].$$

Because the grid dynamics are deterministic, this reduces to maximizing over the single successor state induced by each action. After convergence, a greedy policy is extracted from V^* and rolled out to generate the corresponding path.

4 Results

To evaluate planner behavior under different mission geometries, five test configurations were considered: Baseline, Center Block, Detour Wall, Corridor Sweep, and Circle. Each configuration used the same underlying reward map and detour budget (10), while varying the waypoint arrangement and obstacle layout. The Baseline case provides a simple reference scenario with no obstacles. Center Block introduces a central no-fly region that forces both planners to reroute around a local obstruction. Detour Wall increases the difficulty by placing a larger barrier between waypoint groups. Corridor Sweep extends the mission across a longer path with obstacles that break the map into certain regions. Finally, Circle places waypoints in a loop-like arrangement with a vertical obstacle barrier that separates portions of the route. The purpose of these configurations is to test planner performance under progressively more constrained conditions while keeping the reward map fixed. In this way, differences in total reward and runtime can be attributed primarily to route geometry and obstacle complexity rather than to changes in the underlying reward distribution.

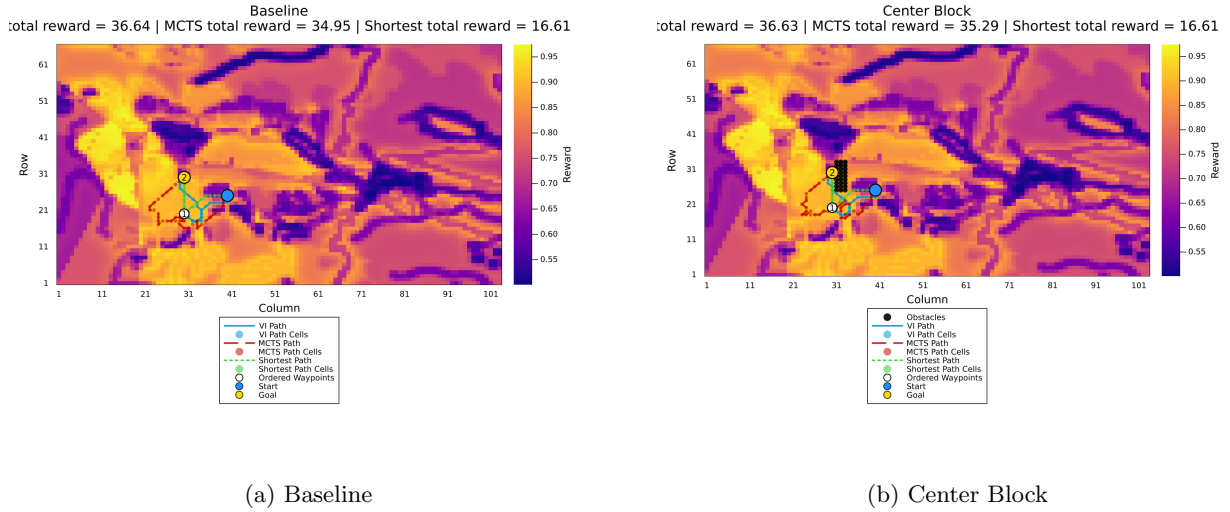


Figure 1: Route comparisons for two representative scenarios.

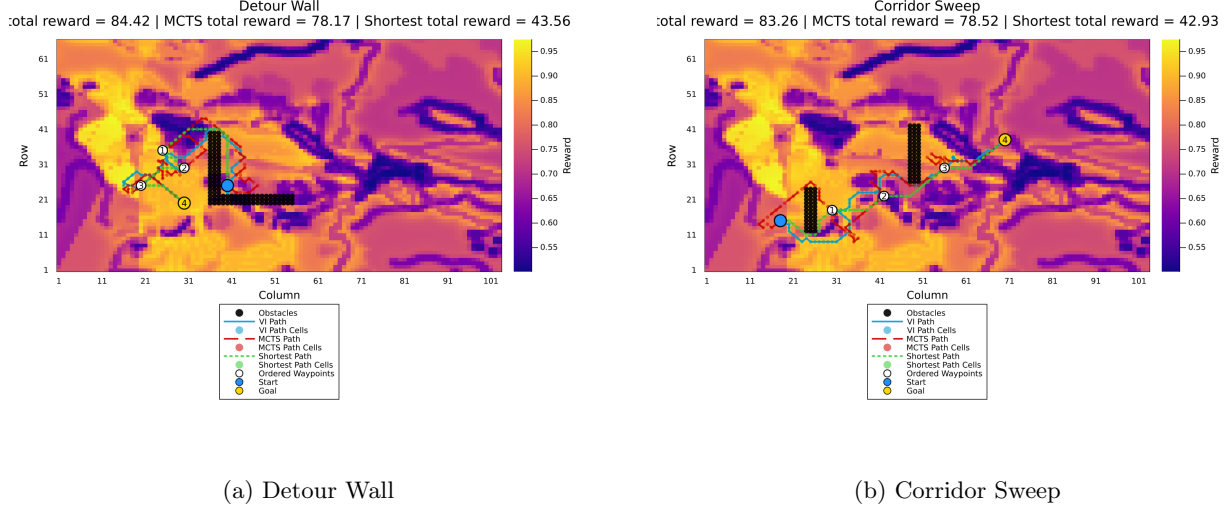


Figure 2: Route comparisons for two representative scenarios.

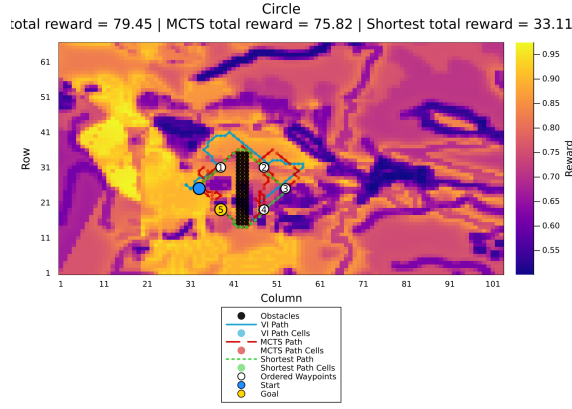


Figure 3: Route comparison for the circle scenario.

Table 1 shows that VI achieved the highest total reward in every test configuration, while MCTS consistently produced the second best solution and substantially outperformed the shortest path baseline. Across the five cases, MCTS achieved between 92.6% and 96.3% of the VI total reward, indicating that it remained close to the exact solution despite using an approximate search procedure. In contrast, the shortest path baseline produced much lower rewards in every case, demonstrating that minimizing travel distance alone is insufficient when the objective is to maximize collected reward over the route.

The strongest advantage of reward aware planning appears when comparing MCTS and VI against the shortest path baseline. In the simpler Baseline and Center Block scenarios, MCTS more than doubled the total reward achieved by shortest path. Similar behavior was observed in the more constrained Detour Wall, Corridor Sweep, and Circle cases, where MCTS still exceeded shortest path by a wide margin. The same trend held for VI, which produced the best total reward per distance overall. These results indicate that both reward aware methods successfully exploit the structure of the reward map, whereas shortest path favors direct routes that miss high value regions.

Case	Method	Reward / Distance
Baseline	VI	0.92
Baseline	MCTS	0.87
Baseline	Shortest Path	0.83
Center Block	VI	0.92
Center Block	MCTS	0.88
Center Block	Shortest Path	0.83
Detour Wall	VI	0.90
Detour Wall	MCTS	0.83
Detour Wall	Shortest Path	0.81
Corridor Sweep	VI	0.88
Corridor Sweep	MCTS	0.83
Corridor Sweep	Shortest Path	0.78
Circle	VI	0.86
Circle	MCTS	0.82
Circle	Shortest Path	0.79

Table 1: Comparison of reward collected per unit distance and runtime for VI, MCTS, and shortest-path planning across the test cases.

5 Conclusion

This work proposes and compares path planning algorithms for drone search and rescue paired with a DRTSP inspired waypoint ordering. It is demonstrated that reward aware planning into waypoint constrained SR navigation significantly improves search performance compared to distance based methods. That is assuming the user defined reward map is an accurate predictor of search performance. Across a variety of tests, it is shown that the proposed VI and MCTS outperform the shortest path in terms of reward per distance. Further, VI is shown to consistently outperform MCTS in terms of reward per distance and runtime. However, MCTS remains close to the near optimal VI in reward collection, indicating that approximate planning methods can still plan effectively in uncertain environments.

The observed performance highlights an important tradeoff between solution quality and scalability. VI performs well for the bounded grid sizes considered because it can systematically evaluate the reachable state space using the known reward map, transition model, and obstacle constraints. However, this advantage may degrade as the map size, waypoint count, or detour budget increases. MCTS is slower in the tested cases, but its sampling-based structure may become more useful in larger environments where exhaustive state evaluation is less practical.

Future work should therefore evaluate these methods on larger reward maps, varied reward distributions, and more complex waypoint and obstacle configurations. Q-learning, included in the Appendix, provides another possible direction because it learns from sampled interaction rather than sweeping over the full state space. However, additional tuning is needed to encourage exploration of high-reward detours while still satisfying waypoint constraints. These extensions would help clarify when approximate or learning-based planners become preferable to VI for larger-scale SAFARI search missions.

6 Contributions

Jack Ryan - Altered abstract for correctness. Adjusted Wavefront Generation algorithm for correctness. Wrote project introduction. Implemented an augmented Q-Learning solver in the SAFARI framework. Wrote conclusion.

Parker Graham - Integrated the value iteration algorithm into the SAFARI system framework. Performed analysis of MCTS, Straight Line, and Value Iteration performance.

Bijan Jourabchi - Integrated waypoint ordering methods and MCTS into SAFARI system. Wrote testing framework and methodology section behind waypoint ordering and MCTS.

References

- [1] Avrim Blum et al. “Approximation Algorithms for Orienteering and Discounted-Reward TSP”. en. In: ().
- [2] Weizhe Chen and Lantao Liu. “Pareto Monte Carlo Tree Search for Multi-Objective Informative Planning”. In: vol. 15. June 2019. ISBN: 978-0-9923747-5-4. URL: <https://www.roboticsproceedings.org/rss15/p72.html> (visited on 04/23/2026).
- [3] Wesley Chorney. “Generalized Wavefront Path Planning”. In: *2025 3rd International Conference on Mechatronics, Control and Robotics (ICMCR)*. Feb. 2025, pp. 25–29. DOI: 10.1109/ICMCR64890.2025.10963265. URL: <https://ieeexplore.ieee.org/abstract/document/10963265> (visited on 04/25/2026).
- [4] Boaz Farbstain and Asaf Levin. “Discounted Reward TSP”. en. In: *Algorithmica* 80.2 (Feb. 2018), pp. 472–495. ISSN: 1432-0541. DOI: 10.1007/s00453-016-0264-2. URL: <https://doi.org/10.1007/s00453-016-0264-2> (visited on 04/21/2026).
- [5] Hunter M Ray, Aditya Pandey, and Nisar Ahmed. “Data Sparse Inference of Operator Spatial Reward Models in Uncertain Environments”. en. In: ().
- [6] Hunter M. Ray et al. *Human-Centered Autonomy for UAS Target Search*. en. arXiv:2309.06395 [cs]. Mar. 2024. DOI: 10.48550/arXiv.2309.06395. URL: <http://arxiv.org/abs/2309.06395> (visited on 07/29/2025).

The authors grant permission for this report to be posted publicly

- **MCTS**: Implemented
- **VI**: Implemented
- **Q-Learning**: Implemented

A Appendix: Goal Oriented Q-Learning Formulation

To explore a scalable alternative to model-based planning, we implemented a Q-learning based solver for waypoint-to-waypoint navigation. Q-learning is a model-free reinforcement learning algorithm that estimates the optimal action-value function $Q(s, a)$ through sampled interaction with the environment. Unlike Value Iteration (VI), which requires repeated sweeps over the full state space, Q-learning updates only the state-action pairs encountered during training episodes, making it a potential candidate for larger planning problems where full dynamic programming becomes expensive.

The state is defined as $s = (p, g)$, where p is the agent’s grid position and g is the cumulative distance traveled. This mirrors the state representation used in MCTS and ensures that feasibility with respect to the detour budget is preserved. The action space consists of the same 8-connected grid motions used throughout the main framework. At each step, actions are restricted to a feasible set defined by the detour budget constraint, ensuring that any selected action still allows the agent to reach the target waypoint.

The Q-learning update follows the standard form:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right],$$

where α is the learning rate and γ is the discount factor. The reward r is taken directly from the SAFARI reward map at the next state, maintaining consistency with the reward definitions used in VI and MCTS. No additional reward shaping or waypoint bonuses are introduced, so performance is evaluated under the same objective as the other planners.

Several modifications were made to adapt Q-learning to the waypoint-constrained planning setting. First, exploration is performed over the set of feasible actions rather than the full action space, ensuring that all sampled trajectories remain reachable under the detour budget. Second, an ϵ -greedy policy is used during training, with a high initial exploration rate that decays gradually over time to encourage broader coverage of the feasible region. Third, during evaluation, actions are selected greedily with respect to $Q(s, a)$, with a tie-breaking heuristic that favors moves reducing the wavefront distance to the waypoint. This helps guide the agent toward the goal when learned values are ambiguous.

This implementation is intended as a lightweight, learning-based baseline rather than a replacement for VI. As discussed in the main text, Q-learning avoids full-state evaluation and may scale more naturally to larger environments, but requires sufficient exploration and tuning to recover high-quality policies. In the current problem setting, where the transition and reward models are known, VI remains more effective, while Q-learning highlights the tradeoffs between model-based and model-free approaches.

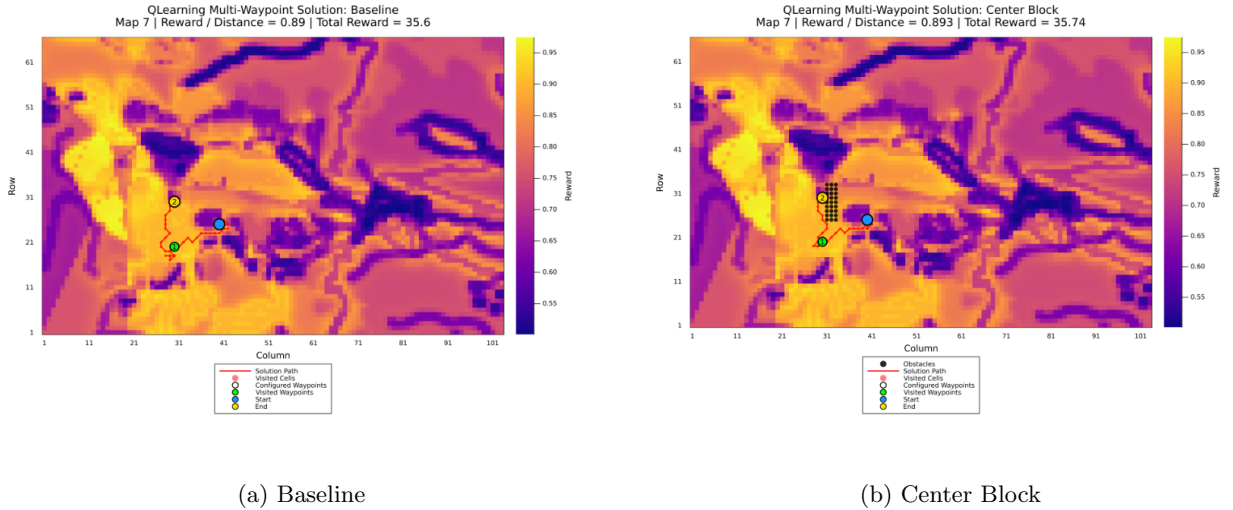


Figure 4: Route comparisons for two representative scenarios.

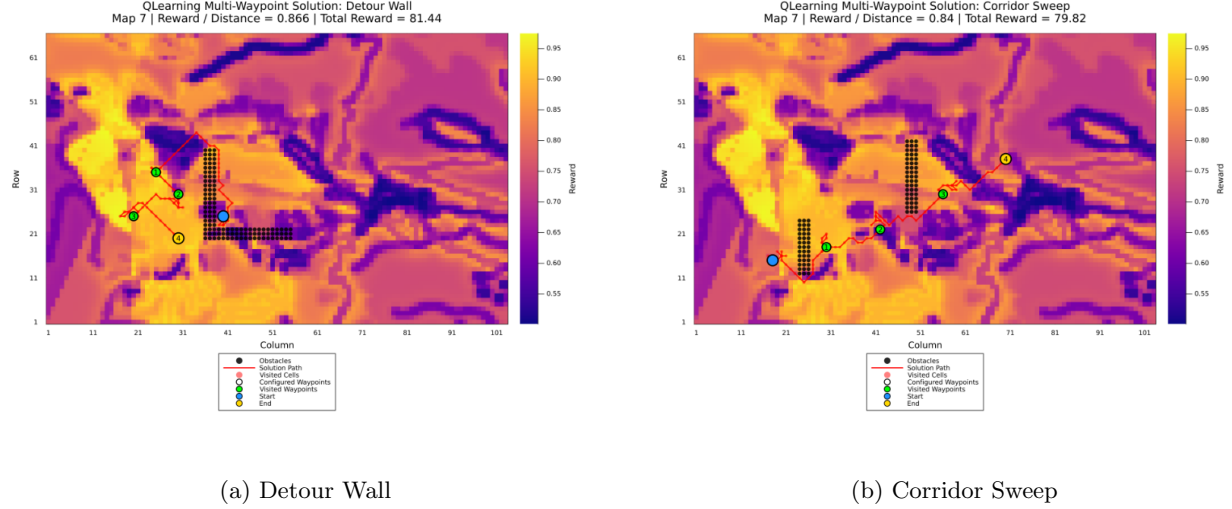


Figure 5: Route comparisons for two representative scenarios.

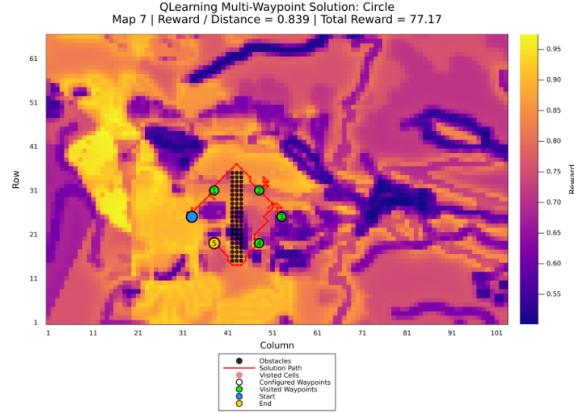


Figure 6: Route comparison for the circle scenario.

Case	Method	Reward / Distance
Baseline	Q-Learning	0.89
Center Block	Q-Learning	0.89
Detour Wall	Q-Learning	0.87
Corridor Sweep	Q-Learning	0.84
Circle	Q-Learning	0.84

Table 2: Reward collected per unit distance for Q-Learning across the test cases.

B Appendix: Q-Learning Results and Discussion

Table 2 shows the performance of Q-learning across the same test configurations used for VI and MCTS. Q-learning consistently outperforms the shortest-path baseline and achieves performance comparable to MCTS in several cases. In the Baseline and Center Block scenarios, Q-learning attains reward-per-distance values of 0.89, slightly exceeding MCTS and approaching the VI benchmark. In more constrained environments

such as Detour Wall, Corridor Sweep, and Circle, performance remains competitive but falls below both VI and MCTS, indicating some loss in efficiency as path complexity increases.

Overall, these results demonstrate that Q-learning is capable of learning reward-aware navigation policies that exploit the structure of the SAFARI reward map while satisfying waypoint constraints. However, unlike VI, which directly computes an optimal policy given the full model, Q-learning relies on sampled interaction and therefore requires sufficient exploration to discover high-reward trajectories. This dependence on exploration likely explains the performance gap observed in more complex scenarios, where optimal paths require coordinated detours that are less frequently encountered during training.

Despite this, Q-learning offers a key advantage in scalability. Because it updates only visited state-action pairs rather than performing full sweeps over the state space, it is expected to scale more favorably to larger maps, higher waypoint counts, and increased detour budgets. In such settings, the computational cost of VI may become prohibitive, while Q-learning can continue to improve policies through incremental learning.

Future improvements to the Q-learning implementation should focus on enhancing exploration of the feasible state space. In particular, exploration strategies that better leverage the detour budget—such as reward-weighted sampling or adaptive exploration schedules—may improve the algorithm’s ability to discover high-value detours. Additionally, incorporating more structured exploration policies or hybrid approaches that combine learning with heuristic guidance may further close the performance gap with model-based methods.

C Appendix: Wavefront Generation

Algorithm 3 Wavefront Generation

Require: g (Goal), $\mathcal{W} \leftarrow \mathbf{0}$ (Wavefront Grid), $\mathcal{O}$ (Obstacles)

```

1:  $\mathcal{W}[o] = 1 \ \forall \ o \text{ in } \mathcal{O}$ 
2: Visited  $\leftarrow \emptyset$  ▷ Empty list of visited cells
3: while  $w$  in  $\mathcal{W}$  not Visited do
4:   for Neighbors of  $w$  do
5:     if Neighbor  $\notin$  Visited and Neighbor  $\notin \mathcal{O}$  then
6:       Add Neighbor to Visited
7:        $\mathcal{W}[\text{Neighbor}] = \mathcal{W}[w] + 1$  ▷ Increment wavefront distance
8:     end if
9:   end for
10: end while
11: return  $\mathcal{W}$ 

```
